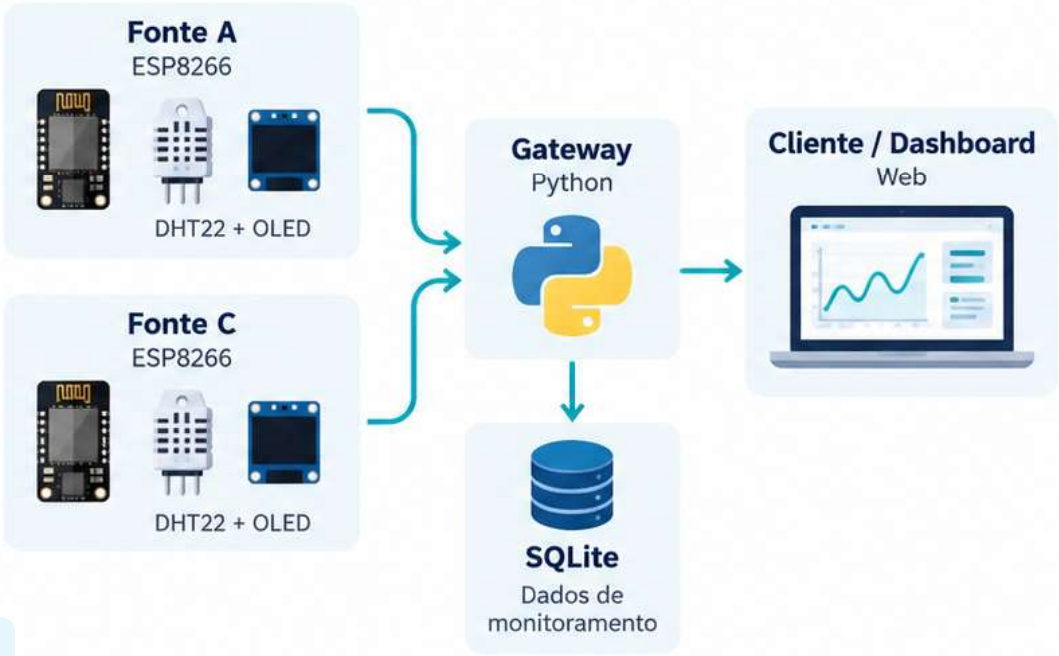
Trabalho 01 – Sistema Distribuído de Monitoramento Ambiental

Arquitetura, implementação e comparação com as exigências do trabalho

Sistemas Distribuídos • Apresentação de 5 minutos

ESP8266 + DHT22 + OLED • Gateway Python • Dashboard Web



Objetivos

1. Mostrar a arquitetura desenvolvida
2. Destacar os principais pontos da codificação
3. Comparar a solução com os requisitos do enunciado

Vagner Henrique | Matrícula: 603486

Requisitos do trabalho × solução implementada

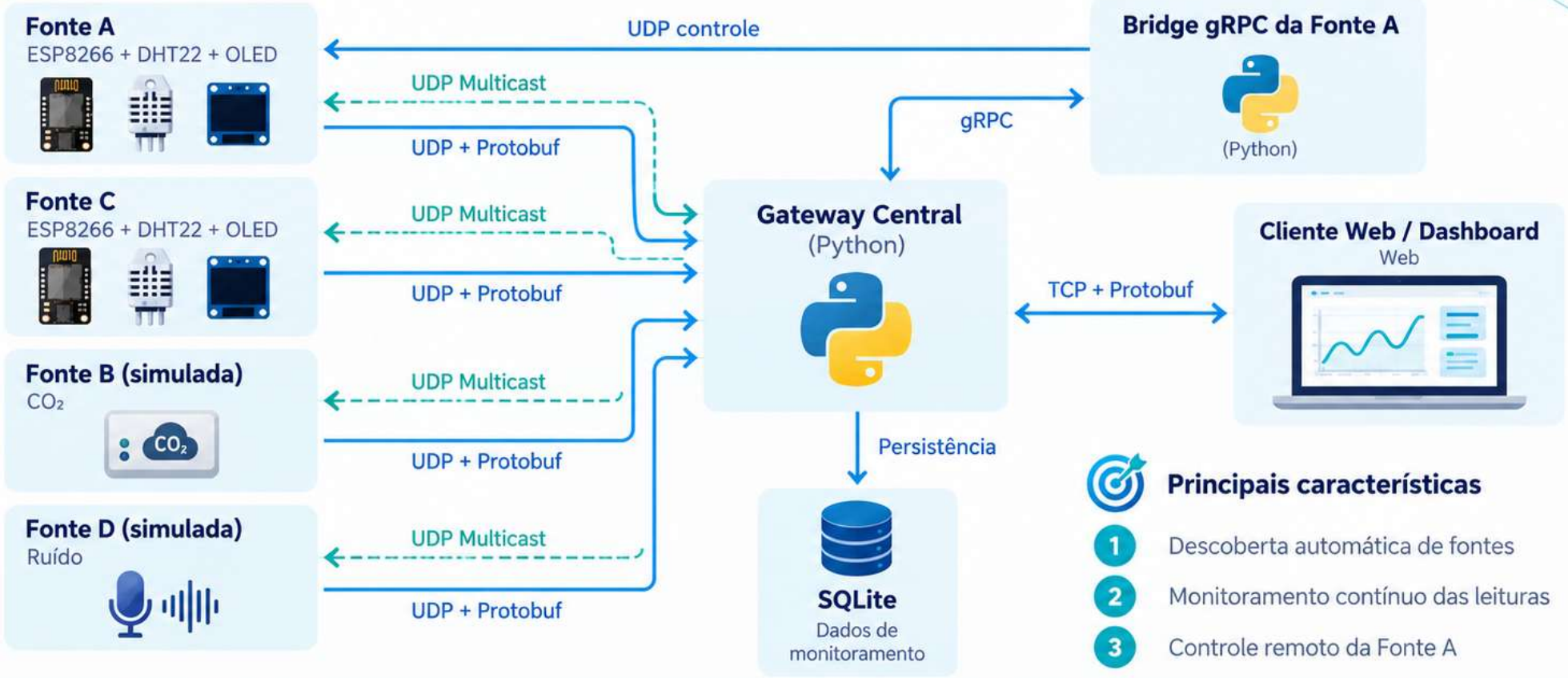
Exigência	Implementação	Situação
Comunicação Fonte → Gateway via UDP	ESP8266 envia telemetria UDP	✓ Atendido
Cliente ↔ Gateway via TCP	Cliente Web/local consulta o Gateway por TCP	✓ Atendido
Uso de Protocol Buffers	Mensagens de telemetria, consulta e controle em Protobuf	✓ Atendido
Descoberta por multicast UDP	Gateway envia WHO_IS_THERE e as fontes se anunciam	✓ Atendido
gRPC para Fonte A	Controle implementado via bridge gRPC por limitação do ESP8266	! Atendido com adaptação
Múltiplos tipos de fonte	Fonte A, Fonte C e fontes simuladas adicionais	✓ Atendido
Persistência de dados (extra)	SQLite no Gateway	✓ Implementado

Observação importante

A Fonte A física é controlada por um bridge gRPC no computador, preservando a funcionalidade exigida pelo trabalho.

Trabalho 01 – Sistema Distribuído de Monitoramento Ambiental

Arquitetura desenvolvida



Principais características

1. Descoberta automática de fontes
2. Monitoramento contínuo das leituras
3. Controle remoto da Fonte A

Trabalho 01 – Principais pontos da codificação

Firmware ESP8266



SensorDHT

Leitura do DHT22/AM2302

```
float t = dht.readTemperature();
float h = dht.readHumidity();
return {t, h};
```



DisplayOLED

Exibe sensor e estado da rede

```
oled.clear();
oled.setCursor(0,0);
oled.println("Temp: 25.3 C");
```



NodeNetwork

Wi-Fi, descoberta e envio UDP

```
WiFi.begin(ssid, pass);
udp.beginPacket(broadcastIP, 5555);
udp.write(payload);
udp.endPacket();
```



ProtoWire

Serialização Protobuf

```
Telemetry msg;
msg.set_temperature(t);
msg.set_humidity(h);
payload = msg.SerializeAsString();
```

Gateway e Cliente



Gateway

Recebe anúncios e telemetria

```
sock = socket.socket(...)
data, addr = sock.recvfrom(1024)
processar_telemetria(data, addr)
```



SQLite

Persistência do histórico

```
conn.execute(
    "INSERT INTO dados (...)
    VALUES (?, ?, ?)",
    (ts, temp, hum))
```



fonte_a_bridge

Traduz gRPC → UDP para a Fonte A

```
req = fonte_pb2.Comando()
req.acao = "LIGAR"
enviar_udp_fonte_a(req)
```



Dashboard Web

Consulta, gráficos e comandos

```
dados = api.get("/dados")
plotar_grafico(dados)
api.post("/comando", {"acao": "ON"})
```

Coleta → Codifica → Transmite → Armazena → Consulta/Controla

Código organizado em módulos para facilitar manutenção e testes.

Fluxo de funcionamento do sistema

Da descoberta das fontes à visualização e controle dos dados

1. Gateway envia **WHO_IS_THERE**
2. Fonte anuncia **ID, tipo e IP**
3. ESP8266 coleta temperatura e umidade
4. **Telemetria UDP + Protobuf** chega ao Gateway
5. Gateway grava no **SQLite** e atualiza estado
6. **Dashboard** consulta dados e, na **Fonte A**, envia comandos

Fluxo de controle (comandos)

Dashboard → Gateway → gRPC bridge → Fonte A

Destaques da solução

- **Baixa sobrecarga** na telemetria
- **Descoberta automática** na rede local
- **Consultas estatísticas** e histórico

Trabalho 01 – Dashboard final e conclusão

Visualização completa do sistema e considerações finais

O que o dashboard oferece

1. Visão geral das fontes
2. Leituras de temperatura e umidade
3. Consultas e histórico
4. Comandos para a Fonte A

Conclusão

1. A solução atende aos requisitos principais do trabalho.
2. A arquitetura integra firmware, gateway, persistência e interface.
3. O principal desvio é o uso do bridge gRPC para a Fonte A física.

